Cold Chain Fleet Management

Technology Stack Guide: tools and frameworks, local setup first, then GCP

> **How to read this guide.** Build and test everything locally first (free, fast to debug), then deploy the same containers and code to GCP. Each tool below has a one-line reason. Section 4 maps every local tool to its GCP counterpart, and Section 5 lists tools you can safely skip.

1. Recommended stack at a glance

- **Language:** Python (generator, processor, API).
- **Event backbone:** Apache Kafka in KRaft mode, with Kafka Connect for integration.
- **Packaging:** Docker and Docker Compose, the same images locally and on GCP VMs.
- **Cloud services:** Pub/Sub, Cloud Functions, Firestore, Cloud Storage, Cloud Run, Cloud Scheduler, Cloud Monitoring.
- **Optional analytics layer:** DuckDB locally, BigQuery on GCP.

2. Local setup: tools and why

Core pipeline

Layer	Tool	Why
Containers	Docker + Docker Compose	One file starts Kafka, Connect, the UI and databases. The same images later run on GCP VMs, so local work carries over.
Language	Python 3.11+	One language for generator, processor and API, with the best library support for Kafka, GCP and data work.
Kafka client	confluent-kafka	Official client built on librdkafka: fast and reliable. Avoid kafka-python, which is unmaintained.
Event backbone	Apache Kafka (apache/kafka image, KRaft mode)	Durable, replayable event log. KRaft removes ZooKeeper. Run 3 containers locally to demo broker failover.
Integration	Kafka Connect (distributed mode, container)	Moves data out of Kafka without hand-written consumers. For a quick local start, a small Python consumer can stand in for it.
Validation	Pydantic	Type and required-field checks with clear errors. The same model can be reused in the processor and the API.
Stream processing	Python consumer + Functions Framework	The rules are simple threshold checks, so Spark or Flink are unnecessary. Functions Framework runs the exact Cloud Function code locally.
Message broker (cloud path)	Pub/Sub emulator (gcloud beta emulators)	Lets you test the Connect-to-Function path without a cloud account or any cost.
Live state store	Firestore emulator (or Redis / MongoDB)	The emulator keeps the same client code as production. Redis is a lighter alternative for latest-value-per-truck.
Raw archive	Dated local folders (/archive/yyyy/mm/dd) or MinIO	MinIO is S3-compatible object storage that behaves like a GCS bucket.

Serving, visibility and testing

Layer	Tool	Why
API	FastAPI + Uvicorn	Fast, async, automatic docs at /docs, easy JSON and WebSocket or Server-Sent Events.
Dashboard	HTML + JavaScript + Leaflet.js + Chart.js	No build step and free. Leaflet draws the truck map, Chart.js draws temperature trends.
Kafka visibility	Kafka UI (kafbat or provectus image)	Shows topics, partitions, leaders, consumer lag and broker status. Ideal for failover demo screenshots.

Layer	Tool	Why
Testing	pytest + docker stop / start	Unit tests for the alert rules, plus a simple chaos test that kills a broker while data flows.
Data source	Synthetic generator (recommended) or Kaggle dataset + pandas	Synthetic data lets you inject temperature excursions on demand, so demos are repeatable. A Kaggle replay adds realism.
Dev tooling	Git, VS Code, python-dotenv, curl / Postman	Version control, config through .env files, and quick API checks.

Optional extras

Layer	Tool	Why
Local analytics	DuckDB + pandas	Run SQL directly over the archived JSONL or Parquet files on a laptop, with no server.
Metrics	Prometheus + Grafana	Only if you want metric dashboards. Kafka UI is enough for most academic demos.

3. GCP: services and why

Infrastructure (the VM-based part)

Layer	Service	Why
Compute	Compute Engine VMs running Docker	Your design runs Kafka, Connect and the generator on VMs. Gives full control over Kafka configuration.
Kafka storage	Persistent Disk (pd-balanced) + snapshots	Kafka data survives VM restarts, and snapshots give a cheap backup.
Networking	VPC, firewall rules, IAP for SSH, Cloud NAT (optional)	Brokers talk over private IPs, nothing is exposed publicly, and SSH goes through Identity-Aware Proxy.
Identity	IAM + service accounts	Connect VMs authenticate to Pub/Sub and GCS without key files, using least-privilege roles.
Secrets (optional)	Secret Manager	Holds any credentials or API keys instead of hard-coding them.

Data pipeline (managed services)

Layer	Service	Why
Messaging	Pub/Sub + dead-letter topic	Managed, multi-zone broker that decouples Kafka from processing. Dead-letter topic catches messages that keep failing.
Processing	Cloud Functions gen2 (Cloud Run functions) + Eventarc	Event-triggered and autoscaling, with no servers to manage and pay-per-use pricing.
Live state	Firestore (Native mode)	Low-latency document per truck, real-time listeners, and a free tier that covers a small simulation.
Archive	Cloud Storage + lifecycle rules	Cheap, durable raw storage. Lifecycle moves data Standard to Nearline to Coldline to Archive automatically.
Serving	Cloud Run (FastAPI container)	Scales to zero, highly available and HTTPS by default. Firebase Hosting is optional for static files.
Scheduling	Cloud Scheduler	Triggers the silence-detection function every minute, so a dead truck does not look healthy.

Operations and delivery

Layer	Service	Why
Observability	Cloud Monitoring, Cloud Logging, alerting policies, Ops Agent	Alert on Pub/Sub oldest unacked message age, Function errors and VM health. Send pipeline alerts through a separate channel from cargo alerts.
Build and deploy	Artifact Registry, Cloud Build, gcloud CLI	Store and build container images, then deploy from the command line.
Infrastructure as code (optional)	Terraform	Repeatable setup and one-command teardown. Looks strong in a report.
Cost control	Billing budgets and alerts	A budget alert at around $10 prevents surprises.
Analytics (optional)	BigQuery + Looker Studio	SQL over the archive and charts such as excursions per truck per day. The free tier covers this volume.
Production alternative	Managed Service for Apache Kafka	Removes VM operations for Kafka. Not needed for an academic project, but worth naming as the production target.

4. Local to GCP mapping

The goal is that moving to the cloud changes configuration, not code.

Component	Local	GCP	What changes
Kafka	Docker containers	Compute Engine VMs running the same containers	bootstrap.servers only
Kafka Connect	Container or Python consumer	Connect workers on the Kafka VMs	Connector configs
Pub/Sub	Pub/Sub emulator	Pub/Sub	Remove PUBSUB_EMULATOR_HOST
Processing	Functions Framework	Cloud Functions gen2	Nothing in the code
live_status	Firestore emulator	Firestore	Remove FIRESTORE_EMULATOR_HOST
Archive	Folder or MinIO	Cloud Storage	Sink connector settings
Dashboard / API	Uvicorn	Cloud Run	Nothing in the code
Silence check	cron or manual curl	Cloud Scheduler	Nothing in the code
Monitoring	Kafka UI	Cloud Monitoring and Logging	Add alert policies

5. Tools to skip, and why

Tool	Why skip it for this project
Spark, Flink, Dataflow	Overkill for per-message threshold rules at this volume, and they add setup and cost. Mention them as the scale-up path in your report.
Airflow / Cloud Composer	Composer is an always-on, costly environment. Cloud Scheduler is enough for the jobs you have.
Kubernetes / GKE	Heavy operations and cost for only four services. Docker on VMs and Cloud Run are simpler.
ZooKeeper	Replaced by KRaft in modern Kafka. It is one more thing to run and fail.
kafka-python library	No longer maintained. Use confluent-kafka.
Confluent GCS connector	Requires a paid license after the trial. The Aiven GCS sink is open source.
Paid managed Kafka (Confluent Cloud)	Adds cost. Name it as the production option instead.

6. Suggested adoption order

1. Docker Compose with Kafka and Kafka UI, then the Python generator and a console consumer.

2. Add the Pub/Sub and Firestore emulators with the Functions Framework processor.
3. Add the archive writer (folder or MinIO) and the FastAPI dashboard.
4. Run the 3-broker failover test locally and capture screenshots.
5. Move to GCP: VMs for Kafka, Connect and the generator, then the managed services.
6. Optional: add DuckDB or BigQuery for analytics, and Terraform for repeatable setup.